

Reinforcement Learning Snake Agent

Q-Learning Implementation and State Optimization

Author: Olivia Martínez

Course: Machine Learning

University: University Carlos III of Madrid

Year: 2025

Phase 1

Selection of the state information and reward function

1. Select the attributes to be used to represent a game state. This information should be generic and not linked to particular board sizes, as it is intended that the snake plays well in as many boards as possible. Also, the fewer attributes you select the better, and if they are nominal with few values the better, since the size of the Q-table will depend on this selection.

Since the size of the Q-table depends on the number of attributes and their possible values, I have opted for a compact and generic representation that facilitates learning in different board sizes. The fewer attributes and values each attribute has, the smaller the Q-table will be and the faster the agent will learn, therefore I searched for the fewer attributes combination.

We have selected the following attributes:

1. The current direction of movement (up, down, left, or right).
2. The relative position of the food (up, down, left, or right).
3. Whether there is danger when moving forward (boolean value).

This combination generates a total of $2 \times 4 \times 4 = 32$ possible states. This design aims to strike a balance between simplicity, learning speed, and the ability to make useful decisions in the game environment.

To implement it, I saved the information in `current_direction`. Then I created a method called `direction_of_food`, which returns the direction of the food with respect to the snake. Finally, for the danger, I created a method called `will_die` which is a boolean that returns whether the snake will die in the next movement. It checks if it collides with its body or gets out of frame.

2. The tuples state and next state have the same attributes, but represent different ticks.

3. Design a reward function that allows the snake to maximize the score of a game.

Since the main objective of our agent is to survive as long as possible while also maximizing the score by eating food, I have designed a reward function that balances these two key aspects: survival and progress toward the food.

To achieve this, I implemented a logic that:

1. Strongly rewards the agent for eating food (+15), as this increases the game score.
2. Strongly penalizes death (-20), to discourage the agent from making risky or suicidal decisions.

3. Rewards the agent for moving closer to the food (+1) and penalizes it for moving away (-1), encouraging strategic movement toward the goal.
4. Introduces a penalty if too many steps pass without eating (-5 if it exceeds 100 steps without food). This prevents the snake from wandering indefinitely around the board without making progress toward the food.

To implement this logic, I added a new variable called `steps_without_food`, which counts how many consecutive steps the snake has taken without eating. This variable resets each time new food is consumed.

Phase 2

Generation of the agent

1. Implement the code of the missing functions `calculate_reward`, `get_state` and `update_q_table`. Also, complete the main loop missing parts in `SnakeGame.py`. In AulaGlobal, you can download the template code for the Q Learning algorithm. You have to apply the changes in `q_learning.py`, `SnakeGame.py` and `snake_env.py`. The objective is to implement the state corresponding to the proposed solutions, updating the Q table and reward function.

The `get_state` method is designed to return the current state of the game in a format that will be able to be used by the reinforcement learning agent. It collects the information about the game that I described in the previous phase, including the snake's current direction, the direction to the food, and whether there is danger ahead. To do so, it predicts the snake's next coordinates based on its current movement and checks whether continuing in the same direction would result in a collision or cause the snake to hit a wall. Specifically, it calls the `will_die` method to determine if the snake is in danger of dying if it continues moving forward. The `get_state` method returns a tuple containing the current direction, the direction of the food, and a boolean indicating whether danger is ahead. This allows the agent to make informed decisions about the next move in the game.

The auxiliary `direction_of_food` calculates the food's relative direction, and `will_die` checks for potential obstacles or boundary violations.

The `calculate_reward` function calculates and returns the reward based on the current game state. It first checks if the game is over, returning a penalty of -20 if so. If the snake eats food, it grants a positive reward of 15, and the food is set to respawn. If not, it calculates the distance to the food before and after the move. The reward is adjusted based on whether the snake moved closer or farther from the food, with a reward of +2 for getting closer and a penalty of -1 for moving farther away. This helps guide the agent towards eating the food efficiently. I have updated the values of this function multiple times to ensure that I get consistent reward values and balanced results.

2. Implement the agent functionality of the tutorial 4 in this new code so that a Q-table is built and updated according to the Q-Learning algorithm. This table will have as many rows as the number of states you have decided to use, and as many columns as actions the snake can execute. For example, if I select two attributes to represent the state: the relative position of the food with respect to the snake, and its distance, I have a nominal attribute with 8 possible values (up, left, right,..., up, left) and I have a numerical attribute whose range will depend on the size of the biggest board I play in. Let's suppose that this range goes from 1 to 10, where 10 is the maximum distance the food can be. Then, the number of rows in our Q table will be $8 \times 10 = 80$ rows, which are all the possible combinations of values of those two attributes. Therefore, you can see that the larger the number of attributes and the range of these, the larger the size of the Q-table. Therefore, it is recommended to have few attributes and to have a manageable size of the Q-table that favors code debugging and learning speed but it's always your work to select the best possible combination of attributes. Don't be afraid of testing different combinations and select the optimal one.

Once I implemented the code for the missing functions and started running the algorithm, I observed that the snake was indeed learning gradually, but it was receiving a very negative reward. I attributed this behavior to a poorly formulated reward function. In our initial implementation, I heavily penalized the snake for going more than 100 steps without eating, as I feared this could result in a snake with good survival skills but no focus on seeking the apple. However, I realized that this penalization was too heavy because it subtracted 5 points from the reward for each tick of the game, resulting in very negative rewards. After analyzing this, I decided to remove it, which resulted in more stable reward values. Furthermore, to ensure that the snake didn't lose focus on the apple, I decided to increase the reward a bit more when it got closer to the food, so that it would balance out the fact that the goal wasn't just survival, but also achieving its main objective: eating the apples.

3. Select the best value for the α , ϵ and γ attributes.

1. The learning rate (α)

As I know if the learning rate is low, the agent will learn slowly because the changes in the Q-table will be small. While, if the rate is high, the agent will learn faster, but it could also become unstable, adjusting too quickly and losing the ability to generalize. I am going to try with different values in order to obtain the best one which better balances efficiency with stability.

In this case, I am going to increase the learning rate to 0.5. This will allow the agent to update the Q-table more quickly and, therefore, learn more efficiently. However, it could also make the agent more prone to making mistakes if it learns too quickly or overfits to past experiences. As the common values of alpha round 0.01 and 0.5 I tried with these extreme values and some others but it was with the default value (0.1) when the snake did its best performance, learning maybe not as quickly as I wished but in a very stable way. This rate balances efficiency with stability.

2. Exploration Rate (ϵ)

As I know, if ϵ is high, the agent will explore more, making random decisions and learning about new possibilities. On the other hand, if ϵ is low, the agent will exploit its current knowledge more often, focusing on the best-known actions. I am going to experiment with different values of ϵ to find the optimal balance between exploration and exploitation.

We tested higher values for the exploration rate, but since it relies on randomness, not only does the snake fail to learn efficiently and quickly, but it also ends up with a very low reward. For this reason, I can conclude that the optimal value is the predefined one.

3. Discount Factor (γ)

As I know a high value for γ means the agent will prioritize long-term rewards, while a low value will make it focus on immediate rewards. I will experiment with different values of γ to find the best one for our agent's learning. The typical values for γ are between 0.8 and 0.99. I will test these extreme values to see how they affect the agent's decision-making and performance over time.

We conducted several tests with different values for the gamma parameter, but I concluded that the optimal value is the predefined one, 0.9. When using lower values, I noticed that the snake seemed to 'forget' its goal of eating apples and surviving in the short term, which led it to get trapped in repetitive loops without making effective progress.

Some conclusions of this first iteration:

The model was trained and evaluated on different board sizes to observe the snake's performance under varied conditions. On smaller boards, such as 100x100, the snake averaged 3.3 apples eaten per episode when the growing body setting was enabled. On larger boards like 500x500, the average improved to 4.26 apples, as the increased space reduced the chances of food spawning near the borders—a common cause of death. The best performance was observed on square boards, such as 250x250, whereas rectangular boards with more extreme proportions, like 450x150, showed slightly worse results, with an average of 2.42 apples. This decrease is likely due to the snake having less room to maneuver and avoid collisions.

In terms of quantitative evaluation, the average score over 50 episodes (with training disabled and body growth enabled) was 377.78. When body growth was turned off, the average score increased significantly to 1397.04. Regarding apples eaten, the snake averaged 4.54 apples per episode across 50 runs (training off, body growth on), with some episodes reaching up to 20 apples. When the growing body setting was disabled, the average rose to 16.9 apples per episode, with a maximum of 68 in a single episode. This highlights how much harder the game becomes when the snake's body grows, as self-collision is the most frequent cause of death.

(Explanations for how I calculated these numbers can be found in phase 3).

Phase 3

Enhancement of the agent

1. Once it works with any board size, change the state so it takes into account the snake's body. Consider different alternatives balancing benefits and q-table size.

In our initial implementation of the Snake game with reinforcement learning, the state representation was quite limited, as it only considered the current direction of the snake, the relative direction to the food, and whether there was danger directly ahead. This resulted in a state space with only 32 possible combinations, which significantly limited our agent's ability to learn more complex and safer behaviors.

To improve this representation, I decided to redesign the function (`compute_position`) and include more detailed information about the environment. The new version of this function not only takes into account the current direction and the relative location of the food, but also whether there is danger in three directions: ahead, left, and right, relative to the snake's current direction. This change allowed us to provide the agent with a more complete perception of its immediate surroundings, without needing to represent the entire game board. As a result, I achieved a state space of 128 possible combinations (4 directions $\times$ 4 food directions $\times$ 2^3 danger combinations), which is still manageable for our Q-table.

To implement this improvement, I added a new method, `check_dangers`, in the `SnakeGameEnv` class, which evaluates whether the snake is at risk of colliding with its body or the edges of the board in any of the three relevant directions. Additionally, I modified the calls to the `compute_position` function in the main training loop, passing the environment as a parameter so it could access this new additional information.

Finally, I updated the value of `number_states` from 32 to 128 to adjust the size of the Q-table to the new number of possible states. With this new representation, I expect the agent to be able to make safer and more efficient decisions, learning policies that not only consider food collection but also collision risks.

2. Train the model and evaluate your agent on as many runs as possible using different board sizes configurations.

The snake performed well on boards of all sizes. I tried boards as small as 100x100, where the average score was 1284.304 and the average number of apples eaten was 14.088, with `growing_body` set to true.

We also tried large board sizes, such as 500x500, which actually performed even better, because it made it less likely that the food spawned close to the border, which is a common cause of death for the snake. In this board, the average score was 2406.758, and the average number of apples eaten was 36.772 (when `growing body = True`).

We noticed that the snake performed best in square boards, such as 250x250, whereas in rectangular boards that had more extreme proportions, such as 450x150, the performance was slightly worse. The average number of apples eaten was 1.532 (when `growing body =`

True). This is likely due to the snake having less room to turn around and effectively avoid collision.

We noticed a sharp contrast with the previous model, with the snake performing significantly better with the new model in all boards except the 450x150 one. In this rectangular board, the snake performed better with the simpler model.

3. Report both quantitative and qualitative results on the agents' behavior.

Quantitative results:

- Average score of 500 episodes (training = False, growing body = True), on a 500x500 board.

To compute this, I implemented the score variable in the same way it was computed in the previous tutorials, to get an idea of how well reinforcement learning performed when compared to the other learning methods. Similarly to the previous tutorials, the snake's score increased by 100 points when it ate an apple, and decreased by 1 point every tick it did not eat. Then I printed the score at the end of each episode.

We also created a variable `total_score` that was not reset to 0 at the beginning of episodes, in order to be able to compute an `average_score` for the whole execution of multiple episodes, in this case, 500. I erased the q-table, trained for about 10000 episodes, then set `training = False` and executed the game for another 500 episodes. For the 500 episodes, the average score was **2406.758**.

If I set `growing body` to False, then the average score was **101654.668**.

- Average number of apples eaten in 500 episodes (training = False, growing body = True), on a 500x500 board.

We implemented a variable called `apples_eaten` that increases by 1 point every time the snake eats an apple. At the end of the episode, I print the number of apples eaten. Similarly to the average score, I displayed the average number of apples eaten for the whole execution of episodes using a variable `total_apples_eaten` that does not reset every episode. After training for 10000 episodes, I set `training = False` to evaluate the performance. Doing so, I can see that in the 500 episodes, the snake eats an average of **36.772** apples per episode. However, there are some particular episodes where the number of apples eaten has reached over 100.

If I set `growing body` to False, then the average number of apples eaten per episode is **1501.106**. The highest number of apples eaten in a single episode is **over 8300**. This is an expected result because the main difficulty of the game comes with growing the body, since collision with itself is the snake's most common cause of death.

Some more qualitative results:

Clearly, the performance of the snake agent has significantly improved with this new model. I very much prefer this improved model because it outperformed the simple model by a large margin, while keeping the Q-table relatively simple (only 128 rows). I also notice how the Q-table scales well to different board sizes, and adapts to varying sets of circumstances.

CONCLUSIONS OF THE PROJECT:

This project aimed to develop a Q-learning agent capable of playing the Snake game effectively across various board sizes. Over three phases, the agent evolved from a simple implementation to a more sophisticated version with improved decision-making and awareness of its environment.

In the first phase, the state representation used just three attributes: the snake's direction, food position, and danger ahead. This resulted in a compact Q-table with only 32 states, allowing for fast learning but limited situational awareness. Early reward functions that penalized slow progress caused instability, so they were later balanced to better encourage both survival and food collection.

Phase 2 introduced the complete agent structure, including key functions like `get_state`, `calculate_reward`, and `update_q_table`. Default hyperparameters provided stable learning. While the agent worked well, eating multiple apples per episode, its limited perception led to errors in complex situations.

The major leap came in Phase 3, where the state was expanded to also consider dangers to the left and right. This increased the state space to 128 but significantly improved the agent's spatial awareness. As a result, the snake better avoided obstacles and achieved higher scores, especially on large or square boards. Tests showed averages of 36.77 apples per episode and scores over 2400 on 500x500 boards.

Although performance on small or irregularly shaped boards varied—sometimes favoring simpler models—the agent overall demonstrated intelligent, learned behavior. It moved efficiently, avoided danger, and consistently pursued food, all with a simple Q-learning algorithm.

In summary, the project met its goals and highlighted how improvements in state representation and reward design can drastically boost performance, even with classic reinforcement learning methods.